

Claude for STEM Professors

From zero to a deployed course app: the accounts, the tokens, and three projects you can put in front of students.

You've probably seen the demo where someone types a sentence and an app appears. The demo skips the account setup. That setup is most of this guide: about an hour, done once. If you can follow a recipe and copy-paste, you can do all of this.

Version: 21 Aug 2026, the day I checked every click-path against the vendor docs. Buttons move. If something isn't where I say it is, check the linked docs.

Latest version: github.com/pisanuw/claude-for-stem-professors (<https://github.com/pisanuw/claude-for-stem-professors>). Run `./build-pdf.sh` if you want a PDF version.

The three destination projects (details in Part 6):

1. **A course website**, built and deployed to Netlify in under an hour
2. **A practice-problem app** for your subject, auto-graded, students use it before exams
3. **A Canvas assistant** that pulls your roster and assignment data and drafts announcements, running on Render

What you need

- A computer with a web browser. Only the Part 8 appendix needs more (a terminal).
- About an hour for setup, then an afternoon per project.
- No money. Every service here has a free tier that covers these projects.

- Coffee. I do this with a cortado at 6am, but you do you.

Contents

- Part 1: Set up Claude
 - Part 2: Create your accounts
 - Part 3: Get your tokens
 - Part 4: Connect Claude to your accounts
 - Part 5: Token hygiene
 - Part 6: Three projects
 - Part 7: Scheduled jobs with GitHub Actions
 - Part 8: Appendix, Claude Code
 - Troubleshooting
-

Part 1: Set up Claude

1.1 Create a Claude account

1. Go to claude.ai (<https://claude.ai>).
2. Click **Continue with Google** and pick your Google account.
3. Done. You're on the Free plan.

1.2 Free vs Pro

Free is a real working tier. The difference is mostly capacity.

	Free (\$0)	Pro (\$20/month, \$17 if annual)
Usage	Limited budget that resets every 5 hours	At least 5x Free, plus a weekly cap
Chat, web search, file uploads	Yes	Yes
Projects, Artifacts, file creation	Yes	Yes
Connectors (Part 4)	Directory connectors, plus one custom connector	Directory plus custom connectors
Model choice	Current default model	Full model picker, including the strongest models
Claude Code (Part 8)	No	Yes
Advanced Research	No	Yes

Start on Free and upgrade the first time the limit stops you mid-thought. Pricing changes often, so check claude.com/pricing (<https://claude.com/pricing>). If you do go Pro, my referral link (<https://claude.ai/referral/X0jtHxAOEA>) throws a small thank-you my way. Feel free to ignore it.

1.3 Turn on network egress (do this in a browser)

One setting is easy to miss. **Allow network egress** lets Claude's code environment reach the internet to install libraries: a plotting package, a PDF parser, whatever the job needs. Without it, most data work fails. Turn it on.

Where: claude.ai (<https://claude.ai>) in a web browser, then **Settings**, then **Capabilities**, then scroll to the **very bottom** of the page. The phone app doesn't show this setting at all; set it once in a browser and it applies to your whole account, phone included.

The security warning next to the setting isn't boilerplate: code that reaches the internet installs software written by strangers, and a token pasted into that chat could in principle be sent somewhere. That's why Part 5 nags about rotating tokens. Have a look at what Claude proposes before approving, the way you'd skim a script a student emailed you.

Part 2: Create your accounts

Three accounts, all free, all reachable with the **Continue with Google** button. Ten minutes total.

Which Google account? Your UW account: one login you already use daily, and the .edu address qualifies you for GitHub's education benefits. Use a personal account only if the work is personal (consulting, a book, anything you'd rather not explain to a chair) or if campus IT blocks third-party sign-ins.

2.1 GitHub

GitHub stores your code and its full history. Every project in this guide lives in a GitHub repository.

1. Go to github.com/signup (<https://github.com/signup>).
2. Click **Continue with Google**.
3. Pick a username. It's public and hard to change, so choose something you'd put on a syllabus, such as your NetID plus `uw`.
4. Complete the verification code GitHub sends you.

While you're in there, two things at github.com/settings/security (<https://github.com/settings/security>): **set a password or passkey** (accounts created through Google have none, so no way in if Google ever locks you out) and **turn on two-factor authentication**.

2.2 Netlify

Netlify hosts websites and frontend apps: your course site, your quiz app. The free tier is fine for a class.

1. Go to app.netlify.com/signup (<https://app.netlify.com/signup>).
2. Click **Sign up with Google**.
3. Answer or skip the onboarding questions. You land on an empty dashboard. Good.

2.3 Render

Render hosts backends: apps with a server, a database, or a secret to keep (like your Canvas token in Project 3). Free services sleep when idle and take a moment to wake, which is fine for a personal tool (render.com/docs/free (<https://render.com/docs/free>)).

1. Go to dashboard.render.com/register (<https://dashboard.render.com/register>).
 2. Click the **Google** button.
 3. Confirm your email if asked.
-

Part 3: Get your tokens

A token is a password with a narrower job: it lets a program act as you on one service without ever seeing your real password. The pattern is always the same: generate, hand to Claude in a chat, let Claude work, delete or rotate (Part 5 has the rules). Treat a token like a password. That's what it is.

Every token below is shown to you **exactly once**, at creation. Copy it into a password manager before closing the page.

3.1 GitHub classic token

The workhorse: it lets Claude create repositories, push code, and set up deploy pipelines.

1. On github.com (<https://github.com>): profile photo (top right), **Settings**, **Developer settings** (bottom of the left sidebar), **Personal access tokens**, **Tokens (classic)**. Direct link: github.com/settings/tokens (<https://github.com/settings/tokens>).
2. Click **Generate new token**, then **Generate new token (classic)**. GitHub may ask you to re-authenticate.
3. **Note**: name it for its job, e.g. `claude-course-projects`.
4. **Expiration**: 90 days. Don't pick "no expiration"; future-you will forget this token exists.
5. **Scopes**: check the entire **repo** box and **workflow**. Nothing else.

6. Click **Generate token** and copy it (starts with `ghp_`). Only time you'll see it.

When it expires, a new one takes two minutes and the old prompts still work.

3.2 Netlify personal access token

You may not need this one. The Netlify connector in Part 4.2 uses a browser sign-in instead, and that's the better path.

1. On app.netlify.com (<https://app.netlify.com>): avatar, **User settings**, **Applications**, then **New access token**. Direct link: app.netlify.com/user/applications (<https://app.netlify.com/user/applications>).
2. Name it, set an **expiration date**, generate, copy once.

One quirk: reset your Netlify password and every existing token dies with it.

3.3 Render API key

1. On dashboard.render.com (<https://dashboard.render.com>), open **Account Settings**, then **API Keys**. Direct link: dashboard.render.com/u/settings (<https://dashboard.render.com/u/settings>).
2. Click **Create API Key**, name it, copy once.

A Render key has broad access to your whole account with no way to narrow it. Prefer the browser sign-in route in Part 4.3 when you can.

3.4 Google OAuth client (for app logins)

The other credentials let Claude act as you. This one lets *other people* prove who they are to an app you built. Project 3 uses it so your Canvas dashboard asks for a Google sign-in.

1. At console.cloud.google.com (<https://console.cloud.google.com>), create a project (top-left project selector, **New project**), name it for the app, e.g. `canvas-dashboard`, and **switch to it**, which is the step everyone forgets.

2. Open **Google Auth Platform** in the left menu and run the **Get started** wizard: app name, support email.
3. **Audience** is the one real decision. **Internal** limits sign-in to your own Google Workspace organization; pick it if it's available. If it's greyed out, pick **External**, which starts in testing mode where only listed test users can sign in. Fine for a dashboard with one user.
4. Under **Data access**, add only `openid`, `email`, and `profile`. These are non-sensitive scopes, so no verification review. Ask for more and you inherit a review process you don't want.
5. If you chose External, add yourself under **Test users**.
6. Open **Clients**, **Create client**, type **Web application**. Anything else gives you the wrong OAuth flow.
7. Under **Authorized redirect URIs**, add both, adjusting the hostname:
 - `http://localhost:5000/auth/callback`
 - `https://YOUR-APP.onrender.com/auth/callback`
8. Click **Create**. Copy the **Client ID** (ends in `.apps.googleusercontent.com`) and the **Client secret**; the console shows only the secret's last four characters afterwards.

The client ID isn't sensitive. The client secret is: rule 6 in Part 5. Google login proves who a user is; it doesn't decide who gets in. Project 3 uses an `ALLOWED_EMAILS` list for that.

3.5 Canvas access token

A Canvas token carries **your full Canvas permissions** through the official Canvas API, which for an instructor includes student names, submissions, and grades. That's FERPA-protected data. Rule 7 in Part 5 is about this token; read it first.

1. In your institution's Canvas: **Account** (profile picture), **Settings**, scroll to **Approved Integrations**, click **+ New Access Token**.
2. **Purpose**: name the task, e.g. `claude-announcement-drafts`. Set an **expiration date**.
3. Click **Generate Token** and copy it once.

No Approved Integrations section? Your institution restricted self-service tokens; ask LMS support. You'll also need your **Canvas base URL**: the address in your browser when you use Canvas, usually `https://yourschool.instructure.com` or a custom domain like `https://canvas.uw.edu`.

3.6 Resend API key (optional, for sending email)

Skip this unless an app needs to send email: a weekly digest, magic-link logins in Project 3. Sending email from a program looks trivial and is not: mail servers are suspicious of new senders, and a hand-rolled sender lands in spam or nowhere. Resend does the unglamorous part.

1. Sign up at `resend.com` (`https://resend.com`). Google and GitHub sign-in both work.
2. Open **API Keys**, click **Create API Key**, name it, give it **Sending access** only.
3. Copy the key (starts with `re_`). Shown once.

You can send from `onboarding@resend.dev` straight away, though it looks like a test address to readers and spam filters alike. Sending from your own address means a domain you own, with SPF and DKIM records added under **Domains** (`@uw.edu` isn't yours to configure). Free tier: 3,000 emails a month, capped at **100 a day**, and the daily cap is the one that bites.

Before you email students from an app you built: send to yourself first. Every time.

3.7 Supabase project (optional, for storing data)

Skip this unless your app needs to remember something after the browser closes; Projects 1 and 2 as written don't. Supabase is a Postgres database with a web dashboard and an API in front of it.

1. Sign up at `supabase.com` (`https://supabase.com`). GitHub and Google both work.
2. Create an organization (Free plan), then **New project**.

3. Name it, set a **database password** (save it in your password manager now, shown once), pick the region closest to your students: `West US` for the Pacific Northwest.
4. Wait a couple of minutes, then open **Settings, API**, and note the **Project URL** and two keys.

The two keys aren't interchangeable, and mixing them up is the classic Supabase mistake:

- The **anon** key (also called publishable) is designed to sit in front-end code where anyone can read it. Fine by design, and only by design.
- The **service_role** key (also called secret) bypasses every access rule you write. Server-side only, environment variable only. If it lands in a page, a repo, or a shared chat, rotate it immediately.

Row Level Security is what actually protects the data. With RLS off, the anon key lets anyone who views your page read and write every row. Tell Claude to enable RLS on every table and write explicit policies, then check it yourself under **Authentication, Policies**. A table with no policy and RLS off is a public spreadsheet with extra steps.

Free tier as of August 2026: 500 MB of database, two active projects, no backups, and **projects pause after about a week without database activity**. That last one ruins teaching tools: you build a quiz app in October, students hit it at 11pm the night before the December final, and the project has been asleep for six weeks. If the app has to be up on a date you care about, resume and check it a few days ahead, or set up the keep-alive job in Part 7. If the data matters, ask Claude for a CSV export script and run it now and then.

On student data: a database is one more third party holding whatever you put in it. The privacy questions from rule 7 in Part 5 apply with more force; named submissions deserve a word with your privacy office first.

Supabase also has a connector (Part 4) if you'd rather not paste keys into chats.

Part 4: Connect Claude to your accounts

Two routes, and this guide uses both.

Connectors are set up once in Claude's settings: you sign in through your browser (OAuth), no token changes hands, and you can revoke access with one click. Use these when they exist. Netlify and Render both have one.

Pasting a token in chat works for everything else. Cruder but universal, and you control the blast radius by controlling the token. GitHub and Canvas use this route. Cleanup is rule 4 in Part 5.

4.1 Where connectors live

- Open claude.ai/customize/connectors (<https://claude.ai/customize/connectors>), also reachable via **Settings > Connectors**.
- Inside a chat, connectors are toggled per conversation: the **+** (or sliders) button at the lower left of the message box, then **Connectors**.

The Free plan includes directory connectors plus **one** custom connector, which is exactly enough for this guide.

4.2 Netlify (official connector)

1. On the Connectors page, find **Netlify** in the directory and click **Connect**.
2. Sign into Netlify in the browser window that opens and click **Authorize**. That's the whole setup. No token.

If Netlify isn't listed in your directory, add it as a custom connector with the URL `https://netlify-mcp.netlify.app/mcp` (on Free that spends your one custom-connector slot, so Render in 4.3 drops back to its API key). Test it in a new chat: *"Using the Netlify connector, list my Netlify sites."* An empty list is the correct answer right now.

4.3 Render (custom connector)

Per Render's docs (<https://render.com/docs/mcp-server>):

1. On the Connectors page, click **Add custom connector**.
2. **Name:** `render` . **URL:** `https://mcp.render.com/mcp` .
3. Open **Advanced settings** and set **OAuth Client ID** to `claude` .
4. Click **Add**, then **Connect**, and approve in the browser window.
No API key needed.

First prompt in any chat that uses it: *"Set my Render workspace to [YOUR WORKSPACE NAME]"* (top left of the Render dashboard).
Then try *"List my Render services."* Prefer not to use a connector?
Paste the API key from 3.3 into the chat instead.

4.4 GitHub (paste the token)

No connector needed. In any chat, hand Claude the classic token from 3.1 along with the task:

Here is my GitHub token: `ghp_XXXXXXXX`

Create a public repo called `test-drive` under my account, add a README that says hello, and send me the link. Do not put the token in any file.

That last line is belt-and-braces. Say it anyway.

4.5 Canvas (paste the token)

Same pattern, plus the base URL:

My Canvas base URL is `https://yourschool.instructure.com`
(`https://yourschool.instructure.com`) and here is my Canvas token: `XXXX`

List my active courses with their course IDs. Read-only for now; do not change anything.

Start read-only. Once you trust the setup, allow writes one action at a time.

Part 5: Token hygiene

Seven rules. All seven together take less time than cleaning up one compromised account.

1. **One token per purpose**, named for the job. Never reuse a token across unrelated projects.
 2. **Always set an expiration.**
 3. **Tokens never go in files.** Not in code, not in a README, not anywhere in a repo. Tell Claude this explicitly every time it builds something. Deployed apps keep secrets in the hosting service's environment variables (Project 3 shows how).
 4. **Delete or rotate after a heavy session.** A token pasted into a chat sits in that conversation's history afterwards: delete the conversation when the work is done, or rotate the token. Pick one and actually do it. Rotation costs two minutes.
 5. **If a token leaks, revoke first, investigate second.** Every service above has a delete button next to the token list.
 6. **An OAuth client secret is a token too.** Environment variable, never the repo. If it leaks, rotate it in the Google console under **Clients**.
 7. **Canvas tokens outrank the other rules.** They reach student data. Check your institution's policy on student data in external tools, AI included: an institutional Claude for Education account may be covered by a data agreement; a personal account is not. When in doubt, ask, or practice on a sandbox course (Instructure's free "Free-for-Teacher" accounts exist for exactly this). Delete the token the moment the task is done; regenerating takes a minute. Your institution's policy beats anything in this guide.
-

Part 6: Three projects

Each project has a starter prompt. Paste it into a fresh chat, fill in the CAPITALIZED bits, change whatever you like. Expect some back and forth before it's right. That's how this works.

Every starter prompt ends with "Ask me any questions before you start." Keep that line; I think it's the most useful sentence in this guide. Without it Claude guesses, and the guesses are plausible and wrong in the ways that eat your afternoon: invented office hours, a 10-week schedule, some framework you never asked for. Told to check first, it comes back with three questions and builds the thing you meant.

It runs the other way too. When something confuses you, ask: "Why React here?" "What does deploy actually mean?" You can't wear out its patience, and nobody else is watching.

Project 1: Course website (30 to 60 minutes)

What you get. A public course site: description, schedule, office hours, policies. Version-controlled on GitHub, live on Netlify, updated by asking Claude for edits.

You need. GitHub token (3.1) and the Netlify connector (4.2), enabled in the chat.

Start from last year's syllabus if you have one. Drag it into the chat (Word, PDF, or a Canvas page pasted as text) and let Claude pull the structure out of it. Correcting a draft is much faster than dictating one, and the result sounds like your course because it came from your course. No syllabus handy? The second prompt starts from a blank page.

With a syllabus to upload:

Here is my GitHub token: PASTE_GITHUB_TOKEN

Attached is my syllabus from last year. Build a course website from it for COURSE NUMBER: COURSE TITLE, Autumn 2026.

Pull the course description, objectives, grading breakdown, and policies from the syllabus. Flag anything that looks out of date (old dates, old textbook editions, dead links) instead of copying it forward silently. Rebuild the schedule against the Autumn 2026 calendar below.

Clean and readable, works on phones, no frameworks needed. Create a public GitHub repo called COURSE-NUMBER-website under my account and push everything. Do not put the token in any file.

Then deploy to Netlify using the Netlify connector and give me the URL.

Ask me any questions before you start.

Starting from scratch:

Here is my GitHub token: PASTE_GITHUB_TOKEN

Create a public GitHub repo called COURSE-NUMBER-website under my account.

Build a course website for COURSE NUMBER: COURSE TITLE, Autumn 2026.

Pages: home (course description, instructor, office hours: YOUR HOURS), a week-by-week schedule built on the Autumn 2026 calendar below, and a policies page (late work, collaboration, AI use). Clean and readable, works on phones, no frameworks needed. Push everything to the repo. Do not put the token in any file.

Then deploy the site to Netlify using the Netlify connector and give me the live URL.

Ask me any questions before you start.

Paste this calendar block into either prompt (University of Washington, Autumn 2026):

UW Autumn 2026 quarter calendar:

- Instruction: September 30 (Wednesday) through December 11 (Friday)
- Length: 11 weeks. Note that Autumn is the long quarter at UW; Winter and Spring are 10 weeks.
- Week 1 is a partial week: Wednesday through Friday.
- Finals: December 12 through December 18
- No classes: Veterans Day, November 11 (Wednesday)
- No classes: Thanksgiving, November 26 (Thursday)
- No classes: Native American Heritage Day, November 27 (Friday)

Build the schedule around these dates. Do not schedule content or due dates on the closure days, and do not silently shift material into finals week.

Not at UW? Swap in your own calendar in the same shape, and spell out the length. The eleven-week Autumn quarter is a UW oddity; left to itself Claude assumes a fifteen-week semester and hands you four weeks that don't exist.

Then try. "Change my office hours to Wednesdays 1-3pm and redeploy." "The Thanksgiving week only has two class days, rebalance the topics." Each edit lands in the repo, so you have the full history.

Stretch. Point a custom domain at it, or add your publications page.

Project 2: Practice-problem app (1 to 2 hours)

What you get. An interactive quiz for your subject: students pick answers, get instant feedback and explanations, see a score. No accounts, no server, no student data collected, which keeps both the engineering and the privacy story simple.

You need. Same as Project 1: GitHub token, Netlify connector.

Starter prompt:

```
Here is my GitHub token: PASTE_GITHUB_TOKEN

Create a public GitHub repo called SUBJECT-practice under my account.
Build a single-page practice quiz app for TOPIC (e.g. "equilibrium
problems in first-year chemistry"). Requirements:

- 10 multiple-choice questions with 4 options each. Write the questions
  at the level of AUDIENCE (e.g. "second-year undergraduates"). I will
  review and correct them, so make them your best attempt.
- After each answer: immediately show right/wrong plus a 2-3 sentence
  explanation.
- A running score and a final summary screen with a "try again" that
  reshuffles question order.
- Works well on phones. Store nothing about the user. No token in any file.

Push to the repo, deploy to Netlify with the Netlify connector, give me
the URL.

Ask me any questions before you start.
```

Then try. Paste in your own question bank ("replace the questions with these 20"), or upload an old exam and have Claude build the quiz from those problems instead of inventing new ones. Ask for LaTeX-rendered math if your subject needs it.

Optional: save results with Supabase. Adding a database changes the privacy story, so do it deliberately: anonymous aggregates ("question 7 is missed by 80 percent of attempts") carry almost none of the risk; named per-student results are a different thing entirely. With a project set up per 3.7:

```
Add anonymous result tracking to the quiz app using Supabase.

Store one row per answered question: question id, whether it was correct,
and a timestamp. No names, no emails, no student identifiers, no IP
addresses, and no session id that could link one student's answers
together across questions.

Enable Row Level Security on the table. The anon key may insert rows and
may not read them back. Show me the policy you wrote and explain what it
allows, because I want to check it myself.

Add a separate page for me that shows the percentage correct per question,
sorted worst first.

Ask me any questions before you start.
```

Read the questions before you share the link. Claude writes plausible questions. You still need to check.

Project 3: Canvas assistant (an afternoon)

What you get. A web dashboard, hosted on Render, that talks to the Canvas API: your course list, assignment submission counts, a grade distribution, and draft announcement text you can paste into Canvas. The Canvas token lives in a Render environment variable, never in the code.

You need. GitHub token (3.1), Render connector (4.3), Canvas token and base URL (3.5), Google OAuth client (3.4), optionally a Resend key (3.6) for Step 4. Rule 7 in Part 5 first.

Where this app lives. Free Render services are public: there's no private option, so the URL is reachable by anyone who finds it, and `canvas-dashboard.onrender.com` isn't hard to guess. Step 3 adds the lock.

Step 1, explore in chat first (no app yet, just check the plumbing):

```
My Canvas base URL is https://YOURSCHOOL.instructure.com and here is my
Canvas token: PASTE_CANVAS_TOKEN
```

```
Read-only: list my active courses with IDs, and for course COURSE_ID
list the assignments with due dates and how many submissions each has.
```

Step 2, build and deploy:

```
Now build the dashboard. Here is my GitHub token: PASTE_GITHUB_TOKEN
```

```
Create a private GitHub repo called canvas-dashboard under my account.
Build a small Python (Flask) web app that reads two environment
variables, CANVAS_TOKEN and CANVAS_BASE_URL, and shows: my course list;
per course, assignments with due dates and submission counts; per
assignment, a simple grade distribution chart; and a button that drafts
an announcement summarizing upcoming deadlines (draft only, nothing is
posted to Canvas).
```

```
Leave authentication out for now, I will add Google login next. Do not
add a password login I will have to rip out.
```

```
Never write any token or password into the code or the repo.
```

```
Then use the Render connector to create a free web service from the
repo, set the environment variables (ask me for the values one at a
time), deploy, and give me the URL.
```

```
Ask me any questions before you start.
```

Step 3, put a real lock on it. You need the client ID and secret from 3.4 and your app's URL from Step 2, which is also the hostname for the redirect URI back in the Google console.

Add Google sign-in to the dashboard.

Use the OAuth authorization code flow with these environment variables: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `ALLOWED_EMAILS` (a comma-separated list), and `FLASK_SECRET_KEY` for session signing. Request only the `openid`, `email`, and `profile` scopes.

Rules:

- Every route except the login page and the OAuth callback requires a signed-in session. Do not leave any route open.
- After Google returns the profile, check the email against `ALLOWED_EMAILS`. Anyone not on the list gets a plain "not authorized" page and no data, even though Google authenticated them fine.
- Verify the `email_verified` claim too.
- The callback path must be `/auth/callback` so it matches what I registered with Google.
- Add a sign-out route that clears the session.
- Generate a random `FLASK_SECRET_KEY` for me to paste into Render. It must not be in the code.

Then redeploy to Render, set the new environment variables, and remind me which redirect URI to add in the Google console.

Ask me any questions before you start.

Test in a private browser window: your own account should reach the dashboard, a different Google account should be refused. If the second test lets you in, the allowlist isn't being checked and the app is still wide open. Stop and tell Claude exactly that.

Step 4, optional: magic links instead of Google. Skip this if Google login works for everyone who needs in. It exists for the co-instructor at another institution or the TA the Google button turns away: a one-time sign-in URL emailed to an address you've already approved. You need the Resend key from 3.6. Add it alongside Google sign-in, not instead of it.

Add magic-link login as a second option next to the existing Google sign-in. Use Resend for delivery, with `RESEND_API_KEY` and `MAIL_FROM` as environment variables.

Flow: the user enters an email, and if it is on `ALLOWED_EMAILS` they get a sign-in link. If it is not on the list, show the exact same "check your inbox" message and send nothing, so the page cannot be used to find out who has an account.

Token rules, please follow these exactly:

- Generate with `secrets.token_urlsafe(32)`. Never a counter, a timestamp, or anything derived from the email.
- Store only a SHA-256 hash of the token, never the token itself.
- Expire after 15 minutes.
- Single use: delete it the moment it is redeemed, and issue a fresh session cookie at that point.
- Rate limit to 3 requests per email address per 15 minutes.

Explain in your reply where the tokens are stored and what happens to them on restart, because I want to know if a free-tier restart logs everyone out.

Ask me any questions before you start.

Test the failure cases, not the happy path: use a link twice (the second should be refused), request one for an address not on the allowlist (nothing should arrive, and the page shouldn't say so), and let one sit for twenty minutes before clicking. Worth saying plainly: a magic link in an inbox is a key to your dashboard, which is why the 15-minute expiry matters.

Then try. "Add a page that flags students with no submissions in the last two weeks." "Draft individual nudge emails I can review." Keep the whole thing in draft-and-review mode. The professor clicks send.

Part 7: Scheduled jobs with GitHub Actions

You already have this one: no new account, no token, no server. GitHub Actions runs a script on a schedule in GitHub's infrastructure. Public repositories get unlimited minutes; private ones get a free monthly allowance that a few small cron jobs won't dent. Three uses here:

- **Keeping a Supabase project awake.** A query every few days resets the inactivity clock (3.7), so the quiz app still works in December.
- **A weekly digest.** Pull from Canvas, email yourself a summary through Resend (3.6).
- **Backups.** Export your Supabase tables to CSV on a schedule and commit them: the backup the free tier doesn't give you.

Secrets go in the repository, never in the workflow file: **Settings, Secrets and variables, Actions, New repository secret**. The workflow reads them by name; anyone reading your code sees the name, not the value.

```
Add a GitHub Actions workflow to this repo that runs every three days and
makes one small query against my Supabase project, so it does not get
paused for inactivity.
```

```
Read the credentials from repository secrets named SUPABASE_URL and
SUPABASE_ANON_KEY. Nothing sensitive in the workflow file. Add a
workflow_dispatch trigger too so I can run it by hand to test.
```

```
Tell me exactly which secrets to add and where, and what a successful run
looks like in the Actions tab.
```

```
Ask me any questions before you start.
```

Two gotchas. Scheduled times are approximate and can run late when GitHub is busy, so never schedule anything to the minute. And **GitHub disables scheduled workflows in a repository with no activity for 60 days**, emailing you first. On a keep-alive repo you deliberately ignore, that's exactly the situation, and the job stops right when you've stopped thinking about it. Check the Actions tab at the start of each term.

Part 8: Appendix, Claude Code

Everything so far runs in the claude.ai chat. Claude Code is the same Claude working in a terminal on your own computer: it reads and edits files in a folder, runs programs, and uses git directly. Worth adopting when projects outgrow the chat window or you want the files local. It needs a paid plan (included in Pro, no API key to look after), and it lives in a terminal, which sounds worse than it is. You type English at it, same as the chat.

Install

macOS or Linux, in Terminal:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows, in PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Homebrew users: `brew install --cask claude-code`.

First session

```
mkdir my-first-project && cd my-first-project  
claude
```

On first run, pick **Claude.ai account** and approve in the browser. Then just talk to it: "Build the Project 2 quiz app in this folder." It asks permission before changing files or running commands. Read what it proposes before you say yes, at least until it has earned some trust.

Connect Netlify and Render

Claude Code uses the same connectors, added from the command line:

```
claude mcp add --transport http netlify https://netlify-mcp.netlify.app/mcp  
claude mcp add --transport http --client-id claude render https://mcp.render.com/mcp
```

Then inside a session, type `/mcp` and authenticate each one in the browser. The GitHub token gets pasted into the session the same way as in chat.

Two habits

- Run `/init` once per project folder: Claude Code writes a `CLAUDE.md` describing the project, which future sessions read automatically. Standing instructions ("never commit tokens", "log changes in CHANGES.md") belong there.
- Prefer terminal-free? The **VS Code extension** gives the same tool a graphical home.

Full documentation: code.claude.com/docs (<https://code.claude.com/docs>).

Troubleshooting

Claude cannot install a package, or a data-analysis step fails.

Network egress is off. Turn it on in a browser: **Settings >**

Capabilities, very bottom of the page (Part 1.3). The phone app doesn't show this setting.

Claude built something different from what you meant. You didn't ask it to ask. Add "Ask me any questions before you start" and run it again.

"Bad credentials" or 401 when Claude uses a token. Expired token, missing scope (GitHub needs `repo` and `workflow`), or a stray space in the paste. Generate a fresh one; it's faster than debugging.

Google sign-in fails with `redirect_uri_mismatch`. The URI registered in the Google console and the one your app sends must match exactly: scheme, hostname, port, path, no trailing slash. Paste both into the chat and let Claude compare them character by character. It's always a typo.

Google says the app is unverified. Expected for an External app in testing mode; click through the advanced link. It never applies if you stayed on `openid`, `email`, and `profile`.

A colleague on ALLOWED_EMAILS still cannot get in. On an External app they also need to be under **Test users** in the Google console. Two separate lists, both required.

A scheduled workflow stopped running. Look for the 60-day inactivity banner in the Actions tab (there's a re-enable button). Otherwise check the cron syntax, since a wrong field silently means a schedule you didn't intend.

An app that worked in October is broken in December. A paused Supabase project (about a week of inactivity does it) or a sleeping Render free service. Check the dashboards before you debug any code.

Magic-link emails never arrive. Check the Resend dashboard logs first: they tell you whether the send failed or the mail was delivered and filtered. Mail from `onboarding@resend.dev` lands in spam routinely. Also check the 100-a-day cap, which pauses sending rather than breaking it.

Canvas has no "Approved Integrations" section. Your institution turned off self-service tokens. Ask LMS support; several universities hand them out through a request form instead.

Claude ignores a connector. Enable it for that conversation: **+** button at the lower left of the message box, then **Connectors**, toggle it on. Also confirm it shows as connected on the Connectors settings page.

"Add custom connector" refuses or is missing. The Free plan allows one custom connector; remove an old one or upgrade.

Render app is slow on first load. Free services sleep when idle. Wait a moment and reload.

You hit the usage limit mid-project. It resets on a rolling 5-hour window. Come back after the reset, or take it as your sign about Pro.

Everything worked yesterday and today it does not. Something upstream changed, or a token quietly expired. Paste the error into the chat and ask Claude what broke. It's better at reading its own stack traces than you'd expect.

A token got committed to a repo by accident. Revoke it immediately (the service's token page has a delete button), then generate a new one. Revoking first makes the leaked copy worthless; cleaning git history is optional afterwards.

Official documentation

- Claude plans: claude.com/pricing (<https://claude.com/pricing>) · Connectors: [support.claude.com](https://support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp) (<https://support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp>)
- GitHub accounts and tokens: [docs.github.com](https://docs.github.com/en/account-and-profile/how-tos/account-management/creating-an-account-on-github) (<https://docs.github.com/en/account-and-profile/how-tos/account-management/creating-an-account-on-github>) · github.com/settings/tokens (<https://github.com/settings/tokens>)
- Netlify: [docs.netlify.com](https://docs.netlify.com/api-and-cli-guides/api-guides/get-started-with-api/) (<https://docs.netlify.com/api-and-cli-guides/api-guides/get-started-with-api/>) and the Netlify + Claude page (<https://www.netlify.com/with/claude/>)
- Render: render.com/docs/mcp-server (<https://render.com/docs/mcp-server>) · render.com/docs/free (<https://render.com/docs/free>)
- Canvas API and tokens: Canvas OAuth docs (<https://canvas.instructure.com/doc/api/file.oauth.html>) · managing access tokens (<https://community.instructure.com/en/kb/articles/662901-how-do-i-manage-api-access-tokens-in-my-user-account>)
- Claude Code: code.claude.com/docs (<https://code.claude.com/docs>)

Version 21 Aug 2026. The current version lives at github.com/pisanuw/claude-for-stem-professors (<https://github.com/pisanuw/claude-for-stem-professors>). Maintained by Yusuf Pisan (<https://github.com/pisanuw>), pisan@uw.edu. Corrections and questions: email me or open an issue.

Written for colleagues who suspect this stuff might be useful but have not had a free afternoon to find out. Now you have a map for the afternoon. Let me know how it goes. :-)